

Struktura funkcjonalna systemu Intent Guard

1. Główny przepływ

```
flowchart TD
    USER[Intencja użytkownika] --> AI_TODO[AI generuje TODO]

    CODE[Kod i struktura projektu] --> DSL1[DSL 1: Code Reality]
    GIT[Historia Git i commity] --> DSL2[DSL 2: Change Intent]
    INTENT[Intencja + TODO + tickety + logi AI] --> DSL3[DSL 3: Declared Intent]

    DSL1 --> CONSOLIDATE[Silnik konsolidacji]
    DSL2 --> CONSOLIDATE
    DSL3 --> CONSOLIDATE

    CONSOLIDATE --> DSL4[DSL 4: Consolidated Project State]

    DSL4 --> VALIDATE[Walidacja zgodności]
    VALIDATE --> REPORT[Raport]
    VALIDATE --> GATE[Decyzja bramki]
    VALIDATE --> DOCS[Propozycje aktualizacji Markdown]

    DOCS --> HUMAN[Akceptacja człowieka]
    REPORT --> HUMAN
    GATE --> HUMAN

    HUMAN -->|zatwierdzenie| APPLY[Zastosowanie zmian]
    HUMAN -->|odrzućenie| REWORK[Agent poprawia pracę]
```

2. DSL 1 — stan rzeczywisty kodu

Cel

Opisuje to, co rzeczywiście znajduje się w kodzie, bez zgadywania intencji autora.

Dane wejściowe

- AST;
- Tree-sitter;
- SCIP lub analizatory natywne;
- struktura katalogów;
- importy i eksporty;
- publiczne API;
- graf wywołań;

- testy;
- konfiguracja pakietów;
- manifest projektu.

Główna funkcja

```
def generate_code_reality_dsl(  
    repository: RepositoryContext,  
    changed_files: list[Path],  
) -> CodeRealityDsl:  
    ...
```

Funkcje pomocnicze

```
def detect_languages(repository: RepositoryContext) -> list[Language]:  
    ...  
  
def select_language_adapters(  
    languages: list[Language],  
) -> list[LanguageAdapter]:  
    ...  
  
def parse_source_files(  
    files: list[Path],  
    adapters: list[LanguageAdapter],  
) -> list[ParsedFile]:  
    ...  
  
def extract_symbols(  
    parsed_files: list[ParsedFile],  
) -> list[CodeSymbol]:  
    ...  
  
def extract_imports(  
    parsed_files: list[ParsedFile],  
) -> list[ImportRelation]:  
    ...  
  
def extract_public_api(  
    parsed_files: list[ParsedFile],  
) -> list[PublicSymbol]:  
    ...  
  
def build_call_graph(  
    parsed_files: list[ParsedFile],  
) -> CallGraph:  
    ...  
  
def build_dependency_graph(  
    ...
```

```

    modules: list[Module],
    imports: list[ImportRelation],
) -> DependencyGraph:
    ...

def discover_tests(
    repository: RepositoryContext,
) -> list[TestDefinition]:
    ...

def map_tests_to_symbols(
    tests: list[TestDefinition],
    symbols: list[CodeSymbol],
) -> list[TestRelation]:
    ...

def calculate_code_hashes(
    symbols: list[CodeSymbol],
    graph: DependencyGraph,
) -> CodeHashes:
    ...

```

Przykładowy wynik

```

dslType: code-reality
version: 1

modules:
  - id: dsl-runtime
    path: packages/dsl-runtime
    publicApi:
      - executeContract
      - validateContract

relations:
  - subject: executeContract
    predicate: calls
    object: dispatchAction

  - subject: executeContract
    predicate: calls_before
    object: validateContract
    observed: false

tests:
  - id: runtime-validation-test
    verifies:
      - validateContract

facts:

```

```
validation_exists: true
validation_before_execution: false
```

3. DSL 2 — intencja zmian z historii Git

Cel

Opisuje kierunek ostatnich zmian i prawdopodobną intencję commitów.

Dane wejściowe

- trzy ostatnie commity zmieniające kod;
- dziesięć ostatnich istotnych zmian jako kontekst;
- commit messages;
- diff;
- semantic diff;
- zmienione symbole;
- dodane i usunięte testy;
- powiązane tickety.

Główna funkcja

```
def generate_change_intent_dsl(
  repository: RepositoryContext,
  detailed_commits: int = 3,
  context_changes: int = 10,
) -> ChangeIntentDsl:
  ...
```

Funkcje pomocnicze

```
def load_git_history(
  repository: RepositoryContext,
) -> list[GitCommit]:
  ...

def filter_code_commits(
  commits: list[GitCommit],
) -> list[GitCommit]:
  ...

def select_detailed_commits(
  commits: list[GitCommit],
  count: int = 3,
) -> list[GitCommit]:
```

```

...

def select_context_changes(
    commits: list[GitCommit],
    count: int = 10,
) -> list[GitCommit]:
    ...

def generate_semantic_diff(
    commit: GitCommit,
    code_adapter: CodeAnalyzer,
) -> SemanticDiff:
    ...

def extract_commit_claims(
    commit: GitCommit,
) -> list[CommitClaim]:
    ...

def infer_change_intent(
    commit: GitCommit,
    semantic_diff: SemanticDiff,
    historical_context: list[GitCommit],
) -> InferredIntent:
    ...

def detect_project_direction(
    changes: list[SemanticDiff],
) -> ProjectDirection:
    ...

def detect_reverted_decisions(
    changes: list[SemanticDiff],
) -> list[RevertedDecision]:
    ...

def calculate_change_confidence(
    commit_claim: CommitClaim,
    semantic_diff: SemanticDiff,
) -> float:
    ...

```

Przykładowy wynik

```

dslType: change-intent
version: 1

detailedCommits:
  - revision: abc123
    declaredIntent: add validation before execution

```

```
inferredIntent: introduce contract validator
confidence: 0.82

changedSymbols:
- validateContract
- executeContract

observedChange:
  validation_function_added: true
  runtime_integration_added: false

projectDirection:
  primaryGoal: stronger contract validation
  confidence: 0.76

conflicts:
- commit_claims_runtime_integration
- semantic_diff_has_no_runtime_call_path
```

4. DSL 3 — deklarowana intencja użytkownika i AI

Cel

Opisuje, co użytkownik polecił oraz jak AI przełożyło tę intencję na TODO i kryteria wykonania.

Dane wejściowe

- pierwotne polecenie użytkownika;
- TODO wygenerowane przez AI;
- ticket;
- README;
- dokumentacja architektury;
- `AI-*.md`;
- `logs.txt`;
- kryteria akceptacji;
- wcześniejsze zatwierdzenia człowieka.

Główna funkcja

```
def generate_declared_intent_dsl(
  user_intent: UserIntent,
  project_documents: list[Document],
  agent_activity: AgentActivity,
```

```
) -> DeclaredIntentDsl:  
    ...
```

Funkcje pomocnicze

```
def capture_user_intent(  
    raw_instruction: str,  
) -> UserIntent:  
    ...  
  
def load_active_ticket(  
    repository: RepositoryContext,  
) -> Ticket:  
    ...  
  
def load_generated_todo(  
    repository: RepositoryContext,  
) -> TodoPlan:  
    ...  
  
def parse_acceptance_criteria(  
    ticket: Ticket,  
    todo: TodoPlan,  
) -> list[AcceptanceCriterion]:  
    ...  
  
def load_agent_claims(  
    agent_files: list[Path],  
) -> list[AgentClaim]:  
    ...  
  
def load_agent_logs(  
    log_files: list[Path],  
) -> list[ExecutionLog]:  
    ...  
  
def map_todo_to_user_intent(  
    user_intent: UserIntent,  
    todo: TodoPlan,  
) -> IntentMapping:  
    ...  
  
def detect_missing_intent_fields(  
    intent: UserIntent,  
) -> list[MissingIntentField]:  
    ...  
  
def detect_todo_scope_drift(  
    user_intent: UserIntent,  
    todo: TodoPlan,
```

```

) -> list[ScopeConflict]:
    ...

def normalize_acceptance_criteria(
    criteria: list[AcceptanceCriterion],
) -> list[NormalizedCriterion]:
    ...

```

Przykładowy wynik

```

dslType: declared-intent
version: 1

userIntent:
  goal: validate contracts before runtime execution
  approvedByHuman: true

todo:
  generatedBy: AI
  approvedByHuman: true

criteria:
  - id: AC-01
    description: validation function exists
    requiredEvidence:
      - exported_symbol
      - unit_test

  - id: AC-02
    description: validation happens before execution
    requiredEvidence:
      - call_order
      - integration_test

agentClaims:
  - criterion: AC-01
    status: completed

  - criterion: AC-02
    status: completed

```

5. Opcjonalny DSL dodatkowy — dowody wykonania agenta

Trzy DSL-e są wymagany minimum.

W praktyce rekomendowany jest również osobny DSL dotyczący wykonania agenta, aby nie mieszać deklarowanej intencji z deklaracjami wykonania.

Główna funkcja

```
def generate_execution_evidence_dsl(  
  agent_activity: AgentActivity,  
  test_results: list[TestResult],  
  repository_diff: RepositoryDiff,  
) -> ExecutionEvidenceDsl:  
  ...
```

Zawartość

```
dslType: execution-evidence  
version: 1  
  
claims:  
  - criterion: AC-02  
    agentStatus: completed  
  
commands:  
  - command: pnpm test  
    result: passed  
  
changedFiles:  
  - packages/dsl-runtime/src/index.ts  
  - tests/runtime.test.ts  
  
evidence:  
  - type: test_result  
    criterion: AC-02  
    value: passed
```

Ten DSL jest przydatny, ponieważ:

```
deklarowana intencja  
≠  
deklaracja wykonania agenta
```

6. DSL 4 — skonsolidowany stan projektu

Cel

Łączy informacje z minimum trzech DSL-i i tworzy jeden model do podejmowania decyzji.

Główna funkcja

```
def consolidate_project_state_dsl(  
  code_dsl: CodeRealityDsl,  
  git_dsl: ChangeIntentDsl,  
  intent_dsl: DeclaredIntentDsl,  
  execution_dsl: ExecutionEvidenceDsl | None = None,  
) -> ConsolidatedProjectStateDsl:  
  ...
```

Funkcje konsolidacji

```
def align_entities(  
  source_dsls: list[BaseDsl],  
) -> EntityAlignment:  
  ...
```

Łączy nazwy odnoszące się do tego samego elementu:

```
validateContract  
contract validation  
AC-02  
runtime validation
```

```
def align_acceptance_criteria(  
  intent_dsl: DeclaredIntentDsl,  
  code_dsl: CodeRealityDsl,  
  execution_dsl: ExecutionEvidenceDsl | None,  
) -> list[CriterionAlignment]:  
  ...
```

Łączy każde kryterium z dowodami.

```
def compare_declared_and_observed_state(  
  declared: DeclaredIntentDsl,
```

```
    observed: CodeRealityDsl,  
  ) -> list[IntentMismatch]:  
    ...
```

Porównuje:

co powinno istnieć
vs
co faktycznie istnieje

```
def compare_commit_claims_with_code(  
  git_dsl: ChangeIntentDsl,  
  code_dsl: CodeRealityDsl,  
  ) -> list[CommitMismatch]:  
  ...
```

Porównuje:

co deklaruje commit
vs
co pokazuje semantic diff

```
def compare_agent_claims_with_evidence(  
  intent_dsl: DeclaredIntentDsl,  
  execution_dsl: ExecutionEvidenceDsl,  
  code_dsl: CodeRealityDsl,  
  ) -> list[CompletionConflict]:  
  ...
```

Porównuje:

co agent oznaczył jako wykonane
vs
czy istnieje kod i wymagane dowody

```
def detect_regressions(  
  current_code: CodeRealityDsl,  
  previous_snapshot: ConsolidatedProjectStateDsl,
```

```
) -> list[Regression]:  
    ...
```

```
def detect_direction_drift(  
    declared_intent: DeclaredIntentDsl,  
    git_intent: ChangeIntentDsl,  
    code_reality: CodeRealityDsl,  
) -> list[DirectionDrift]:  
    ...
```

```
def calculate_criterion_status(  
    criterion: NormalizedCriterion,  
    evidence: list[Evidence],  
) -> CriterionStatus:  
    ...
```

Możliwe wyniki:

VERIFIED
PARTIAL
MISSING
CONFLICT
UNKNOWN

```
def calculate_project_gate(  
    criteria: list[CriterionStatus],  
    conflicts: list[Conflict],  
    policy: ApprovalPolicy,  
) -> GateDecision:  
    ...
```

Możliwe decyzje:

ALLOW_CHECKPOINT
REVIEW_REQUIRED
BLOCK_STAGE
BLOCK_RELEASE

7. Przykład DSL 4

```
dslType: consolidated-project-state
version: 1

ticket: ticket-014
stage: runtime-validation

criteria:
  - id: AC-01
    declared: true
    codeEvidence: true
    testEvidence: true
    agentClaim: completed
    humanApproval: pending
    effectiveStatus: REVIEW_REQUIRED

  - id: AC-02
    declared: true
    codeEvidence: false
    testEvidence: partial
    agentClaim: completed
    humanApproval: pending
    effectiveStatus: CONFLICT

conflicts:
  - type: FALSE_COMPLETION
    criterion: AC-02
    severity: BLOCKING
    explanation: >
      Agent marked validation-before-execution as completed,
      but the code graph shows execution before validation.

releaseGate:
  todoUpdate: blocked
  changelogUpdate: blocked
  versionUpdate: blocked
  nextStage: blocked

recommendedAction:
  - reopen AC-02
  - correct runtime call order
  - rerun integration tests
```

8. Główny kontroler paczki

```
def start_session(
    raw_user_intent: str,
    repository_path: Path,
) -> SessionResult:
    repository = inspect_repository(repository_path)

    user_intent = capture_user_intent(raw_user_intent)
    baseline = create_repository_baseline(repository)

    todo = discover_or_generate_todo(user_intent, repository)

    return create_work_session(
        repository=repository,
        user_intent=user_intent,
        todo=todo,
        baseline=baseline,
    )
```

Paczka nie tworzy samodzielnie celu pracy. Cel pochodzi od użytkownika.

AI korzystające z paczki może wygenerować TODO, a paczka rejestruje relację:

```
intencja użytkownika
→ wygenerowane TODO
```

9. Checkpoint po pracy agenta

```
def run_checkpoint(
    session: WorkSession,
) -> CheckpointResult:
    repository_diff = collect_repository_diff(session.baseline)

    code_dsl = generate_code_reality_dsl(
        repository=session.repository,
        changed_files=repository_diff.changed_files,
    )

    git_dsl = generate_change_intent_dsl(
        repository=session.repository,
        detailed_commits=3,
        context_changes=10,
    )
```

```

intent_dsl = generate_declared_intent_dsl(
    user_intent=session.user_intent,
    project_documents=load_project_documents(session.repository),
    agent_activity=load_agent_activity(session.repository),
)

execution_dsl = generate_execution_evidence_dsl(
    agent_activity=load_agent_activity(session.repository),
    test_results=load_test_results(session.repository),
    repository_diff=repository_diff,
)

consolidated_dsl = consolidate_project_state_dsl(
    code_dsl=code_dsl,
    git_dsl=git_dsl,
    intent_dsl=intent_dsl,
    execution_dsl=execution_dsl,
)

gate = evaluate_checkpoint_gate(consolidated_dsl)

return save_checkpoint_result(
    consolidated_dsl=consolidated_dsl,
    gate=gate,
)

```

10. Generowanie dokumentacji

```

def propose_documentation_updates(
    consolidated_dsl: ConsolidatedProjectStateDsl,
) -> DocumentationProposal:
    ...

```

Funkcje:

```

def generate_todo_patch(
    state: ConsolidatedProjectStateDsl,
) -> FilePatch:
    ...

def generate_ticket_patch(
    state: ConsolidatedProjectStateDsl,
) -> FilePatch:
    ...

def generate_module_docs_patch(

```

```

        state: ConsolidatedProjectStateDsl,
    ) -> FilePatch:
        ...

    def generate_project_map_patch(
        state: ConsolidatedProjectStateDsl,
    ) -> FilePatch:
        ...

    def generate_changelog_patch(
        state: ConsolidatedProjectStateDsl,
    ) -> FilePatch:
        ...

    def generate_version_patch(
        state: ConsolidatedProjectStateDsl,
    ) -> FilePatch:
        ...

```

System tylko przygotowuje patche.

```

def apply_approved_proposal(
    proposal: DocumentationProposal,
    approval: HumanApproval,
) -> ApplyResult:
    if not approval.is_valid_for(proposal.evidence_hash):
        raise InvalidApprovalError()

    return apply_patches(proposal.patches)

```

11. Human approval

```

def request_human_approval(
    review_package: ReviewPackage,
) -> ApprovalRequest:
    ...

```

```

def record_human_decision(
    request: ApprovalRequest,
    decision: HumanDecision,
) -> HumanApproval:
    ...

```

Decyzje:


```
APPROVE
REJECT
REQUEST_CHANGES
ACCEPT_EXCEPTION
CHANGE_INTENT
```

Każde zatwierdzenie jest powiązane z:

- haszem intencji;
- haszem kodu;
- haszem dowodów;
- commitem;
- ticketem;
- czasem;
- osobą zatwierdzającą.

12. Komendy CLI

```
intent-guard start
```

Rejestruje intencję, TODO i baseline.

```
intent-guard checkpoint
```

Generuje DSL 1, DSL 2, DSL 3, opcjonalny DSL dowodów oraz DSL 4.

```
intent-guard verify-stage
```

Sprawdza wszystkie kryteria etapu.

```
intent-guard report
```

Generuje raport Markdown i JSON.

```
intent-guard propose-docs
```

Przygotowuje patche dokumentacji.

```
intent-guard release-check
```

Sprawdza, czy można aktualizować changelog i wersję.

```
intent-guard review
```

Przygotowuje pakiet do akceptacji człowieka.

```
intent-guard apply --approval APPROVAL_ID
```

Stosuje zatwierdzone patche.

13. Podział odpowiedzialności

DSL 1

Kod mówi: co rzeczywiście istnieje.

DSL 2

Git mówi: co i w jakim kierunku było zmieniane.

DSL 3

Intencja i TODO mówią: co powinno powstać.

DSL wykonania

Agent i logi mówią: co agent twierdzi, że zrobił.

DSL 4

Konsolidacja mówi: co jest zgodne, czego brakuje i co należy zablokować.

Najważniejsza reguła:

```
Żaden z DSL-i źródłowych nie może sam zdecydować, że zadanie jest ukończone.
```

Końcowy status powstaje dopiero przez:

DSL 1

+ DSL 2

+ DSL 3

+ opcjonalny DSL wykonania

→ DSL 4

→ reguły bramki

→ zatwierdzenie człowieka